

MSC2050 Discrete Mathematics, Presentation 8

Dr. Anna Tomskova

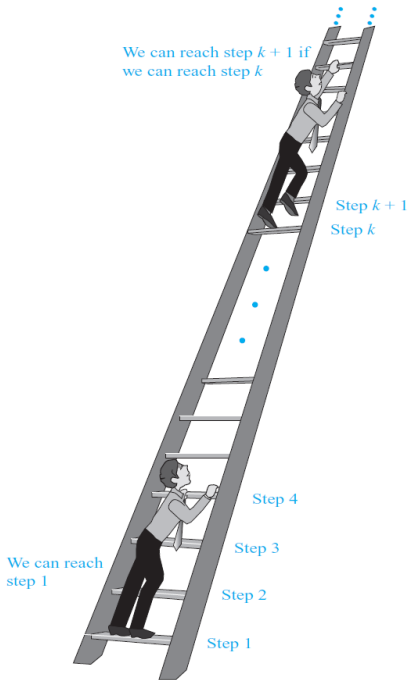


Inha University in Tashkent

Spring 2024
Version 1.0 typed under $\text{\LaTeX}$

Mathematical Induction

(Paragraph 5.1)



1. We can reach the first rung of the ladder.
2. If we can reach a particular rung of the ladder, then we can reach the next rung.

Question: Can we reach every rung?

Conclusion: Yes

PRINCIPLE OF MATHEMATICAL INDUCTION:

To prove that $\forall n P(n)$ is true, we complete **two** steps:

BASIS STEP: $P(1)$ is true.

INDUCTIVE STEP:

$\forall k (P(k) \rightarrow P(k + 1))$ is true.

Proving conjectures

Find the sum of the first n positive odd integers

Solution: For $n = 1, 2, 3, 4, 5$ we have $1 = 1$, $1 + 3 = 4$, $1 + 3 + 5 = 9$,
 $1 + 3 + 5 + 7 = 16$, $1 + 3 + 5 + 7 + 9 = 25$.

It is reasonable to conjecture that the sum of the first n positive odd integers is n^2 , that is,

$$P(n) = "1 + 3 + 5 + \dots + (2n - 1) = n^2".$$

BASIS STEP: $P(1) = "1 = 1^2"$ is true.

INDUCTIVE STEP: We show that $\forall k(P(k) \rightarrow P(k + 1))$ is true.

Let $P(k)$ be true for an arbitrary positive integer k , that is,

$$1 + 3 + 5 + \dots + (2k - 1) = k^2.$$

then for $P(k + 1)$ we have $1 + 3 + 5 + \dots + (2k - 1) + (2k + 1) =$

$$[1 + 3 + \dots + (2k - 1)] + (2k + 1) =$$

$$k^2 + (2k + 1) =$$

$$k^2 + 2k + 1 = (k + 1)^2.$$

Hence, $P(k + 1)$ is true.

By the principle of mathematical induction we can conclude that $\forall n P(n)$ is true.

Proving inequalities

Use mathematical induction to prove the inequality

$$n < 2^n$$

for all positive integers n .

Solution: Let $P(n) = "n < 2^n"$.

BASIS STEP: $P(1) = "1 < 2^1"$ is true.

INDUCTIVE STEP: Assume that $P(k) = "k < 2^k"$ is true for an arbitrary positive integer k .

Then

$$k + 1 < 2^k + 1 < 2^k + 2^k = 2^{k+1}$$

and so $P(k + 1)$ is true.

By the principle of mathematical induction we have shown that $n < 2^n$ is true for all positive integers n .

The Number of Subsets of a Finite Set

Use mathematical induction to show that if S is a finite set with n elements, where n is a nonnegative integer, then S has 2^n subsets.

Solution: Let $P(n)$ = "a set with n elements has 2^n subsets".

BASIS STEP: $P(0)$ is true, because a set with zero elements, the empty set, has exactly $2^0 = 1$ subset, namely, itself.

INDUCTIVE STEP: For the inductive hypothesis we assume that $P(k)$ is true for an arbitrary nonnegative integer k .

let T be a set with $k + 1$ elements.

Then, $T = S \cup \{a\}$, where a is one of the elements of T and $S = T - \{a\}$ (and hence $|S| = k$).

For each subset X of S there are exactly two subsets of T , namely, X and $X \cup \{a\}$.

Since S has 2^k subsets, there are $2 \cdot 2^k = 2^{k+1}$ subsets of T .

By mathematical induction it follows that $P(n)$ is true for all n .

Be careful with the induction

Given a set of n points. Then these points lay on one line.

Proof: Let $P(n) = "n \text{ points lay on one line}"$.

BASIS STEP: Clearly, one point lays on one line so $P(1)$ is true.

INDUCTIVE STEP: Assume that $P(k)$ is true.

Consider a set of $k + 1$ points. Consider a subset of k points. Then these lay on a line. Consider another subset of k points. Then these lay on a line. The intersection of these sets contain $k - 1$ points, so these lines are the same. Hence, $P(k + 1)$ is true.

Flaw:

If $k = 2$, then the intersection is just a point where multiple lines go through.

Homework:

- Solve all odd exercises at p. 329 until #23 and also numbers 31,33,39,41,43,49,51.

Answer Q1-Q2

Strong Induction and Well-Ordering

(Paragraph 5.2)

STRONG INDUCTION: To prove that $P(n)$ is true for all positive integers n , we complete two steps:

BASIS STEP: We verify that $P(1)$ is true.

INDUCTIVE STEP: We show that

$$\forall k (P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1))$$

is true.

Proof of factorization theorem

Theorem

Show that if n is an integer greater than 1, then n is prime or can be written as the product of primes.

Solution: Let $P(n) = "$ n is prime or can be written as the product of primes"

BASIS STEP: $P(2)$ is true.

INDUCTIVE STEP: Let $P(j)$ is true for all integers $2 \leq j \leq k$.

There are two cases to consider, namely, when $k + 1$ is prime and when $k + 1$ is composite.

If $k + 1$ is prime, we immediately see that $P(k + 1)$ is true.

Otherwise, $k + 1$ is composite and so $k + 1 = a \cdot b$ with $2 \leq a, b < k + 1$.

We can write both a and b as the product of primes.

Thus, $k + 1 = a \cdot b$ can be written as the product of primes.

The well-ordering property

The validity of both the principle of mathematical induction and strong induction follows from a fundamental axiom of the set of integers, **the well-ordering property**:

Every nonempty set of non-negative integers has a least element.

The well-ordering property, the principle of mathematical induction, and strong induction **are all equivalent** (and equivalent to the axiom of choice).

Theorem

Prove that if a is an integer and d is a positive integer, then there are integers q and r with $0 \leq r < d$ and $a = dq + r$.

Solution: Let S be the set of non-negative integers of the form $a - dq$, where q is an integer. By the well-ordering property, S has a least element $r = a - dq_0$.

The integer r is non-negative.

It is also the case that $r < d$. Suppose that $r \geq d$. It follows that

$$a - dq_0 - d = a - d(q_0 + 1) = (a - dq_0) - d = r - d \geq 0.$$

So $a - dq_0 - d \in S$, but $a - dq_0$ was the least one. Hence $r < d$.

Homework:

- Solve exercises # 1,3,25,29,37,41 at p. 341-344.

Recursive Algorithms (The Merge Sort)

(Paragraph 5.4)

An algorithm is called **recursive** if it solves a problem by reducing it to an instance of the same problem with smaller input.

Example 1 (power)

```
procedure power( $a$ : non-zero real number,  $n$ : non-negative integer)
if  $n = 0$  then return 1
else return  $a \cdot \text{power}(a, n - 1)$ 
```

Example 2 (gcd)

```
procedure gcd( $a, b$ : non-negative integers with  $a < b$ )
if  $a = 0$  then return  $b$ 
else return gcd( $b \bmod a, a$ )
```

Example 3 (linear search)

```
procedure search( $i, n, x$ : integers,  $1 \leq i \leq n$ )
if  $a_i = x$  then return  $i$ 
else if  $i = n$  then return 0
else return search( $i + 1, n, x$ )
```

Proving Recursive Algorithms Correct

Prove that the algorithm in Example 1 is correct.

Solution: We use mathematical induction on the exponent n with the predicate

$$P(n) = \text{" The algorithm correctly computes } a^n \text{"}$$

BASIS STEP: If $n = 0$, the first step of the algorithm tells us that $\text{power}(a, 0) = 1$. This is correct because $a^0 = 1$.

INDUCTIVE STEP: Assume that $P(k)$ is true, that is $\text{power}(a, k)$ returns a^k , so $\text{power}(a, k + 1) = a \cdot \text{power}(a, k) = a \cdot a^k = a^{k+1}$.

We can conclude that the algorithm always computes a^n correctly.

Time complexity of recursion (power)

Algorithm 1

```
procedure power( $a$ : non-zero real number,  $n$ : non-negative integer)  
if  $n = 0$  then return 1  
else return  $a \cdot \text{power}(a, n - 1)$ 
```

Let $T(n)$ be the time complexity of the algorithm.

$$T(n) = T(n - 1) + C, \quad n > 0, \quad \text{and} \quad T(0) = 1.$$

Then

$$\begin{aligned} T(n) &= T(n - 1) + C = T(n - 2) + 2C = T(n - 3) + 3C = \dots \\ &= T(1) + (n - 1)C = T(0) + nC = O(n). \end{aligned}$$

Algorithm 2

```
procedure power( $a$ : non-zero real number,  $n$ : non-negative integer)
if  $n = 0$  then return 1
else if  $n$  is even
    then  $y := \text{power}(a, n/2)$ 
        return  $y \cdot y$ 
    else return  $a \cdot \text{power}(a, n - 1)$ 
```

Let $T(n)$ be the time complexity of the algorithm.

$$T(n) = \begin{cases} T(n/2) + c_1 & n \text{ is even} \\ T(n-1) + c_2 & n \text{ is odd} \end{cases}, \quad \text{and} \quad T(0) = 1.$$

Then if n is odd we also have

$$T(n) = T((n-1)/2) + c_1 + c_2$$

Hence the even and odd cases are similar.

So consider the case $n = 2^k$,

$$\begin{aligned} T(n) &= T(n/2) + c_1 = T(n/4) + 2c_1 = T(n/8) + 3c_1 = \dots \\ &= T(n/2^k) + kC = T(1) + \log n C = O(\log n). \end{aligned}$$

Time complexity of recursion (search)

Linear search

```
procedure search( $a$ : array of integers,  $n, x$ : integers,  $1 \leq i \leq n$ )  
if  $a_n = x$  then return  $n$   
else if  $n = 0$  then return 0  
else return search( $a, n - 1, x$ )
```

Let $T(n)$ be the time complexity of the algorithm (in the worst case).

$$T(n) = T(n - 1) + C, \quad n > 0, \quad \text{and} \quad T(0) = 1.$$

Then

$$\begin{aligned} T(n) &= T(n - 1) + C = T(n - 2) + 2C = T(n - 3) + 3C = \dots \\ &= T(1) + (n - 1)C = T(0) + nC = O(n). \end{aligned}$$

Binary search

```
procedure search( $a$ : sorted array of integers,  $\ell$ ,  $r$ ,  $x$ : integers)
if  $r \geq \ell$  then  $mid := \lfloor (r + \ell)/2 \rfloor$ 
    if  $a_{mid} = x$  then return  $mid$ 
    else if  $a_{mid} > x$  then return search( $a, \ell, mid - 1, x$ )
    else return search( $a, mid + 1, r, x$ )
else return 0
launch search( $a, 1, n, x$ )
```

Let $T(n)$ be the time complexity of the algorithm (in the worst case).

$$T(n) = T(n/2) + C, \quad n > 0, \quad \text{and} \quad T(0) = 1.$$

So consider the case $n = 2^k$,

$$\begin{aligned} T(n) &= T(n/2) + c_1 = T(n/4) + 2c_1 = T(n/8) + 3c_1 = \dots \\ &= T(n/2^k) + kC = T(1) + \log n C = O(\log n). \end{aligned}$$

Time complexity of recursion (sorting)

Bubble sort

```
procedure bubbleSort(a: array of integers, n: integer)
if n = 1 then return a
else for i := 1 to n - 1
    if  $a_i > a_{i+1}$  then swap( $a_i, a_{i+1}$ )
    bubbleSort(a, n - 1)
```

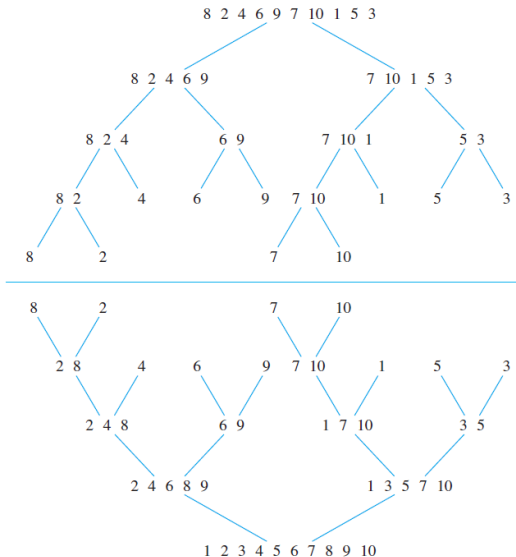
Let $T(n)$ be the time complexity of the algorithm (in the worst case).

$$T(n) = T(n - 1) + Cn, \quad n > 0, \quad \text{and} \quad T(1) = 1.$$

Then

$$\begin{aligned} T(n) &= T(n - 1) + Cn = T(n - 2) + 2Cn = T(n - 3) + 3Cn = \dots \\ &= T(1) + (n - 1)Cn = O(n^2). \end{aligned}$$

Merge sort



A Recursive Merge Sort

```
procedure mergesort( $L = a_1, \dots, a_n$ )  
if  $n > 1$  then  $m := \lfloor n/2 \rfloor$   
     $L_1 := a_1, a_2, \dots, a_m$   
     $L_2 := a_{m+1}, a_{m+2}, \dots, a_n$   
     $L := \text{merge}(\text{mergesort}(L_1), \text{mergesort}(L_2))$   
return  $L$ 
```

Merging Two Lists

```
procedure merge( $L_1, L_2$  : sorted lists)  
 $L :=$  empty list  
while  $L_1$  and  $L_2$  are both nonempty  
    remove smaller of first elements of  $L_1$  and  $L_2$  from its list  
    put it at the right end of  $L$   
    if this removal makes one list empty  
        then remove all elements from the other list and append them to  $L$   
return  $L$ 
```

Answer Q4

Complexity of the Merge Sort algorithm

Let $T(n)$ be the time complexity of the algorithm.

$$T(n) = 2T(n/2) + Cn, \quad n > 0, \quad \text{and} \quad T(1) = 1.$$

Let $n = 2^k$.

Then

$$\begin{aligned} T(n) &= 2T(n/2) + Cn = 2(2T(n/4) + Cn/2) + Cn \\ &= 4T(n/4) + 2Cn = \dots \\ &= 2^k T(n/2^k) + kCn = nT(1) + Cn \cdot \log n = O(n \log n). \end{aligned}$$

Master Theorem (Paragraph 8.3)

Theorem

Let T be an increasing function that satisfies the recurrence relation

$$T(n) = aT(n/b) + cn^d$$

whenever $n = b^k$, where k is a positive integer, $a \geq 1$, b is an integer greater than 1, and c and d are real numbers with c positive and d non-negative. Then

$$T(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Proof is by induction left as an exercise.

Fast Matrix Multiplication

$$T(n) = 7T(n/2) + 15n^2/4$$

is $O(n^{\log_2 7 < 2.81})$

Homework:

- Solve *odd* exercises starting at p. 370.

Answer Q5